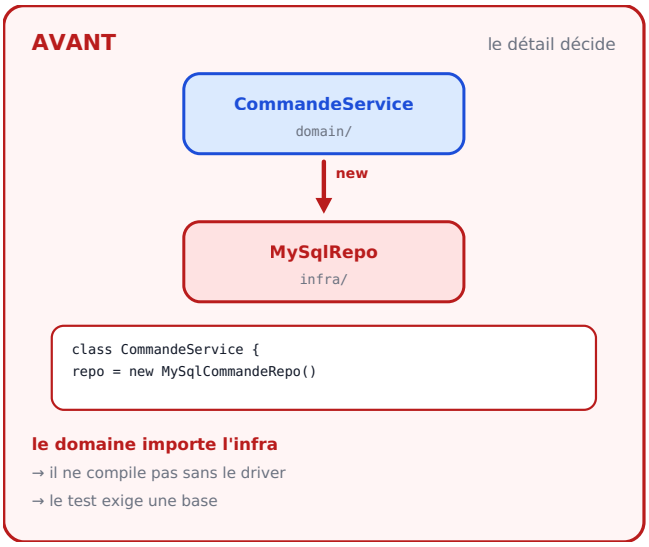
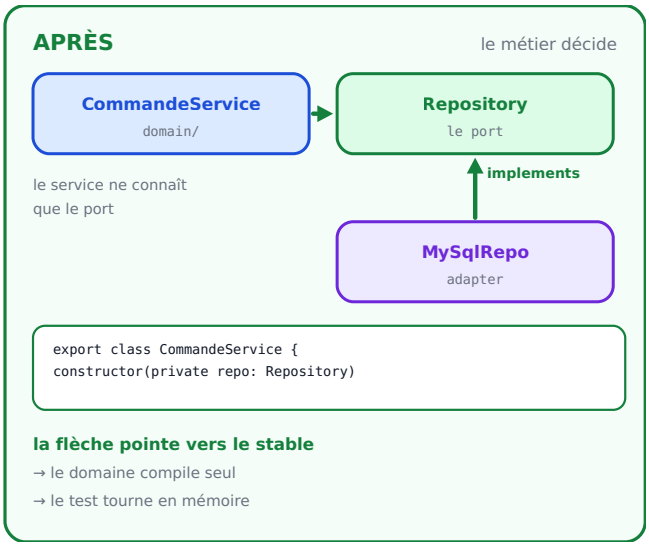
SOLID · LE D

Dependency /Inversion

Le métier décide, l'infrastructure s'y conforme

Tout le principe en schémas : la flèche qui se retourne, la propriété du port, la racine de composition, l'architecture en ports & adaptateurs, et 10 cas réels découpés sous tes yeux.





1	Les deux règles : le sens et la propriété	2
2	La flèche qui se retourne	3
3	Le test : la règle du new, la flèche, le test sans le monde	4
4	Anatomie de la violation	5
5	Anatomie du respect · ports & adaptateurs	6
6	10 cas réels, avant / après en images + code	7 - 11
7	Le reste de la boîte en un coup d'œil	12
8	Les pièges : DIP ≠ conteneur	13
9	Arbre de décision : ce détail mérite-t-il un port ?	14
10	Récap visuel · flashcards	15
11	DIP dans SOLID · la règle de dépendance stable	16
12	Injecter : 3 façons, 5 refactorings	17
13	Exercices corrigés · vrai / faux	18
14	La fiche mémo	19

> Pour réviser : les pages 3 et 6 suffisent — la **flèche** (qui dépend de qui) et les **ports & adaptateurs** (où vit l'interface). Le reste n'est que déclinaison de ces deux idées.

« Le métier ne dépend de rien : on lui branche le monde. »

Dependency Inversion Principle

1 · LES DEUX RÈGLES

Le sens de la flèche, et à qui appartient l'interface

« Les modules de haut niveau ne doivent pas dépendre des modules de bas niveau ; les deux doivent dépendre d'abstractions. »

La phrase de Bob Martin contient deux règles — et c'est la seconde qu'on oublie

RÈGLE 1 · LE SENS



la flèche pointe vers ce qui est stable,
jamais vers ce qui est en bas.

« en bas » n'est pas « stable ».

RÈGLE 2 · LA PROPRIÉTÉ

```
domain/ports.ts ← le port est ICI
infra/mysql-repo.ts ← l'adaptateur est ICI
```

l'interface appartient à celui qui l'utilise,
pas à celui qui l'implémente.

c'est ça, l'inversion.

Inverser ≠ appeler dans l'autre sens : le métier appelle toujours l'infra à l'exécution.

Ce qui s'inverse, c'est le sens de la dépendance de compilation — et la propriété de l'interface.

RÈGLE	FORMULATION COURTE	VIOLATION TYPIQUE
Le sens	la flèche pointe vers ce qui est **stable**	`domain` importe `infra`
La propriété	l'interface appartient au **module de haut niveau**	le port expose les types du SDK (`Stripe.Charge`)
Le détail se conforme	l'implémentation est un **adaptateur**	le SDK fuit dans le service métier
Un seul point de montage	la **racine de composition** connaît les deux	`new` dispersé dans 12 fichiers

AVANT — le métier s'équipe lui-même

```
// domain/commande.service.ts
import { MySqlRepo } from "../infra/mysql-repo"

export class CommandeService {
  private repo = new MySqlRepo()

  valider(id: string) {
    const c = this.repo.trouver(id)
    if (!c) throw new Error("inconnue")
    c.valider()
    this.repo.sauver(c)
  }
}
```

APRÈS — le métier déclare son besoin

```
// domain/ports.ts
export interface CommandeRepository {
  trouver(id: string): Commande | null
  sauver(c: Commande): void
}

// domain/commande.service.ts
export class CommandeService {
  constructor(private repo: CommandeRepository) {}
  // même code, mais plus aucun import d'infra
}
```

Le port parle le fournisseur

```
interface StripePort {
  createCharge(dto: ChargeDto): Charge
  createCustomer(c: CustomerDto)
}
// le métier utilise les mots de Stripe
// changer de fournisseur = réécrire le métier
```

Le port parle le métier

```
interface Paiement {
  debiter(centimes: number, ref: string): Recu
}
// le métier ne sait pas qui débite
// Stripe, Adyen, un faux : un adaptateur
```

Le sens, en clair

Le métier ne dépend pas de l'infra ; les deux dépendent d'une abstraction que le métier possède.

Le test du compilateur

Supprime `infra/` : si `domain/` compile encore, la flèche est bonne.

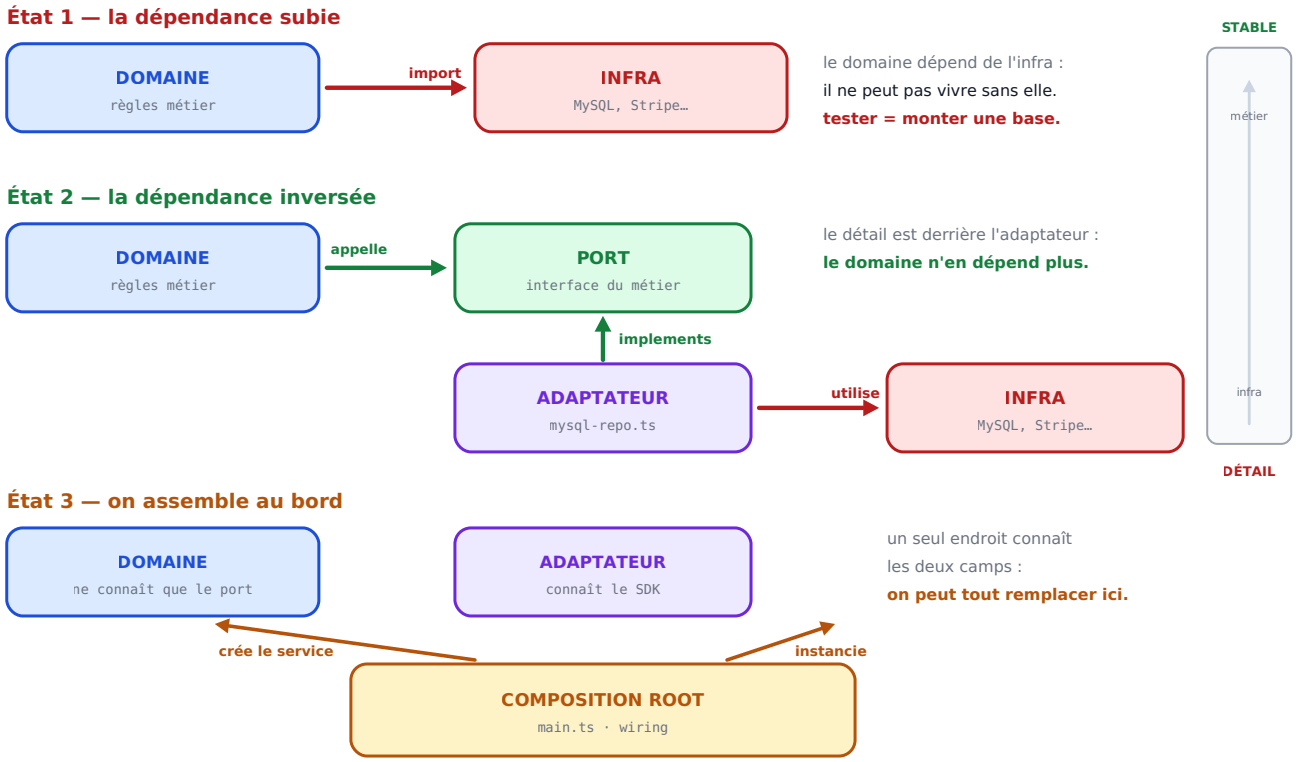
La question du port

« Quel mot du métier décrit ce besoin ? » debiter, trouver, maintenant — jamais « create », « send », « get ».

2 · LA FLÈCHE

Ce qui s'inverse n'est pas le sens de l'appel : c'est le sens de la dépendance

À l'exécution, le métier appelle toujours l'infra. À la compilation, c'est l'infra qui dépend du métier.



Toutes les flèches montent : vers l'abstraction, c'est-à-dire vers ce qui change le moins.

CE QUE FAIT LE MÉTIER	COMPILATION	TEST
il appelle `repo.trouver()`	il ne connaît que l'interface	on injecte un repo en mémoire
il appelle `paiement.debiter()`	il ne connaît pas Stripe	on injecte un faux paiement
il lit l'heure	il ne connaît pas `Date`	on injecte une horloge figée
il émet un événement	il ne connaît pas la file	on lit les événements en mémoire

A · Ce qui ne s'inverse pas

Le **sens de l'appel** : à l'exécution, le métier appelle bien `repo.trouver()`. L'inversion est statique, pas dynamique.

B · Ce qui s'inverse

La **dépendance de compilation** : l'infra compile contre l'interface du métier, et plus l'inverse. C'est une inversion de propriété.

C · Ce que ça débloque

Un domaine qui se compile et se teste seul : c'est la condition de tout le reste (tests rapides, remplacement de fournisseur, lisibilité).

La métaphore du secteur : la prise électrique n'a pas été inventée par EDF mais par les **fabricants d'appareils**. Le standard appartient à celui qui l'utilise — puis le fournisseur s'y branche. C'est exactement le port.

Le piège de la « double interface » : écrire `IStripe` qui recopie les méthodes du SDK ne fait que déplacer le problème — l'abstraction dépend encore du détail. Si l'interface change à chaque montée de version, elle appartient au mauvais camp.

3 · LE TEST

Le domaine doit pouvoir tourner sans le monde

Pas de Docker, pas de réseau, pas d'horloge système : c'est la définition opérationnelle de DIP.

Trois vérifications, du plus rapide au plus profond

1 La règle du new

```
grep -rn "new [A-Z]" src/domain
```

2 La flèche

```
grep -rn "infra" src/domain
```

3 Le test sans le monde

```
vitest run src/domain
```

Ce que le test doit pouvoir faire

- ✓ règle métier sans base — `InMemoryCommandeRepo`
- ✓ date figée — `Clock { maintenant: () => T }`
- ✓ paiement sans réseau — `FauxPaiement`
- ✓ email sans SMTP — `InMemoryNotificateur`

Aucun de ces quatre tests n'a besoin de Docker, d'un port réseau ou de l'horloge système.

Le nouveau est le signal

```
// dans src/domain/ :
new MySqlRepo() ✗
new Stripe(key) ✗
new Date() ✗
crypto.randomUUID ✗
process.env.X ✗

// à la place :
des paramètres du constructeur
```

A · La règle du new

Un new dans `domain/` est une dépendance que le métier s'inflige : elle est invisible dans la signature.

B · La flèche

Si `domain` importe `infra`, la dépendance pointe vers le bas : le principe est violé, même sans new.

C · Le test sans le monde

Un use case qui s'exécute avec des faux en mémoire est la preuve que l'inversion est faite — et qu'elle tient.

Le test, tel qu'il devient possible

```
// valider-commande.test.ts — aucun conteneur, aucun réseau
const repo = new InMemoryCommandeRepo()
const clock = { maintenant: () => new Date("2026-01-01T00:00:00Z") }
const service = new CommandeService(repo, clock)

it("refuse une commande inconnue", () => {
  expect(() => service.valider("nope")).toThrow("inconnue")
})
```

Le test le plus rapide

`grep -rn "new [A-Z]" src/domain` : chaque ligne trouvée est une dépendance que le métier s'inflige.

Le test le plus profond

Supprimer le dossier `infra/` et recompiler : ce qui casse indique exactement où la flèche est mauvaise.

Le test de trop

Un test qui démarre Docker n'est plus un test unitaire : c'est une intégration déguisée.

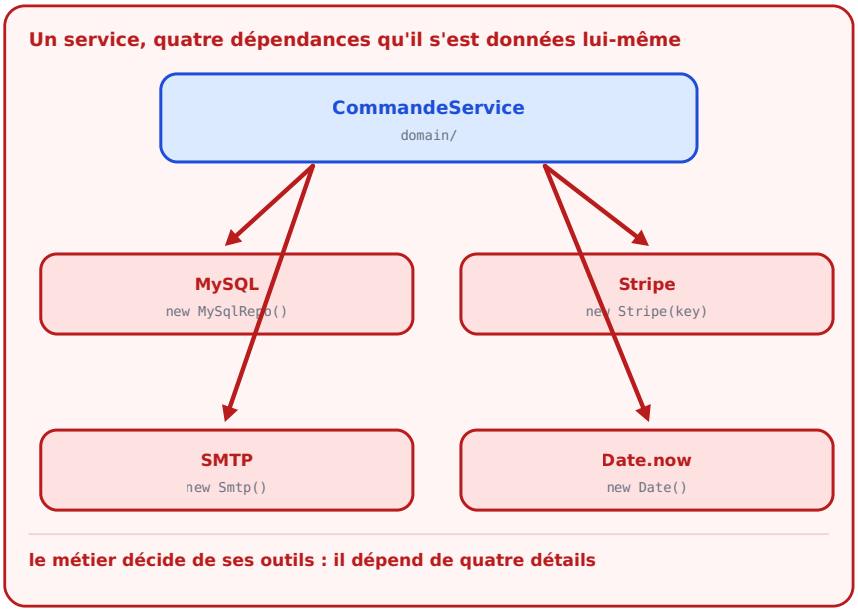
CE QUE LE TEST PROUVE	CE QU'IL NE PROUVE PAS	OÙ LE VÉRIFIER
la règle métier est correcte	que MySQL répond bien	tests d'intégration de l'adaptateur
le port est suffisant pour le use case	que le port est complet	le second adaptateur
le temps et le hasard sont maîtrisés	que l'horloge système est juste	nulle part : c'est la libc
le service est remplaçable	que le câblage est correct	un test de la racine de composition

Le test le plus utile n'est pas celui qui passe : c'est celui qu'on **peut écrire**. Un domaine plein de dépendances subies n'est pas seulement fragile — il devient illisible pour quiconque veut vérifier une règle.

4 · LE DIAGNOSTIC

Anatomie de la violation

Un service qui se donne ses propres outils produit toujours les mêmes trois symptômes.



Ce que ça coûte

```
// pour tester valider() :
1. une base MySQL
2. une clé Stripe
3. un serveur SMTP
4. une date système

// ou 5 mocks à écrire
// → personne ne teste
```

✗ le domaine ne compile pas seul

✗ le fournisseur impose son API

✗ changer de base = chasser le SQL

Le symptôme :

des if (NODE_ENV), des singletons, et des tests qu'on n'écrit jamais.

A · Signal n°1

Un new technique dans le métier : `new MySqlRepo()`, `new Date()`, `new Stripe()`.

B · Signal n°2

Un `import` de SDK ou de driver dans un fichier de règles métier.

C · Signal n°3

Un test qui exige une base, un réseau, une variable d'environnement ou l'horloge réelle.

CE QUE LE CLIENT ÉCRIT	DÉPENDANCE SUBIE	DÉPENDANCE INVERSÉE
le constructeur du service	`constructor()` — rien à injecter	`constructor(repo, clock)`
le test	monter une base + un SMTP	deux objets de 5 lignes
le changement de fournisseur	chasser le SDK dans 30 fichiers	réécrire **un** adaptateur

Détecter une violation en 30 secondes

1 `grep -rn "new [A-Z]" src/domain`

une dépendance qui s'auto-inflige

→ on regarde ce qu'il trouve

2 `grep -rn "infra" src/domain`

la flèche pointe dans le mauvais sens

→ on regarde ce qu'il trouve

3 `grep -rn "process.env" src/`

un détail de déploiement dans le métier

→ on regarde ce qu'il trouve

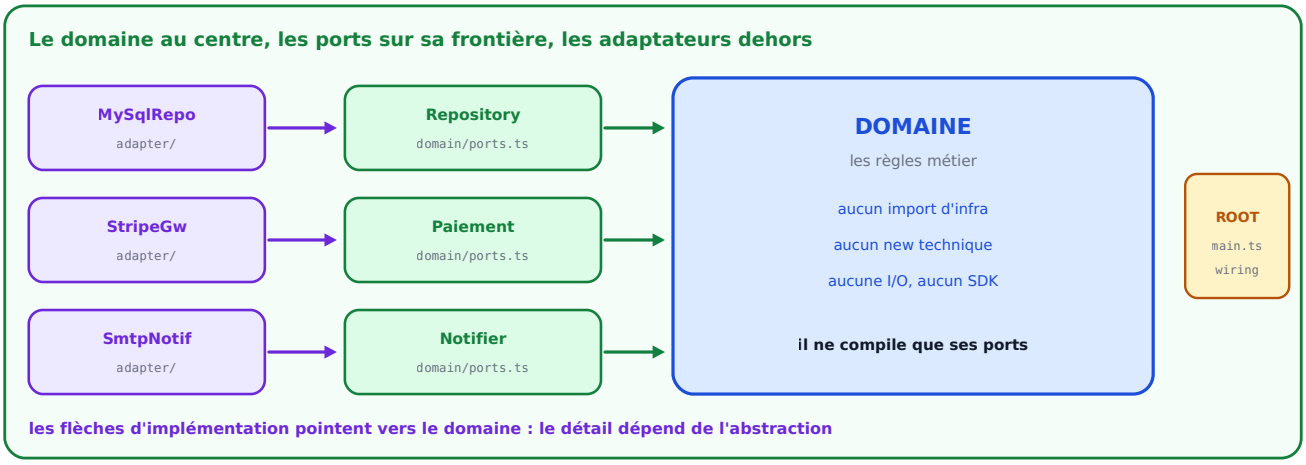
LA DÉPENDANCE SUBIE	LE PORT À ÉCRIRE	L'ADAPTATEUR
`new MySqlRepo()`	`CommandeRepository`	`MySqlCommandeRepo`
`new Stripe(key)`	`Paiement`	`StripeGateway`
`new Smtp()`	`Notificateur`	`SmtpNotificateur`
`new Date()`	`Clock`	`SystemClock`

Le vrai coût : ces quatre dépendances ne coûtent pas un seul bug aujourd'hui — elles coûtent **l'absence de tests demain**. C'est pour ça qu'on ne les voit pas arriver : rien ne casse, on arrête juste de vérifier.

5 · LA CIBLE

Ports & adaptateurs : le domaine au centre

Le métier ne connaît que ses ports ; tout ce qui touche le monde est dehors et s'y branche.



Les 3 gestes, dans cet ordre

1 Extraire le port

écrire l'interface DANS le domaine, avec ses mots : debiter, trouver, maintenant.

2 Pousser l'adaptateur

le SDK ne sort plus du fichier infra/ : il implemente le port.

3 Composer au bord

main.ts (ou wiring) est le seul endroit qui connaît les deux.

Le critère de réussite : supprime le dossier `infra/` et le domaine **compile toujours**. Ajoute un second adaptateur (S3, Stripe, faux) et tu ne modifies **aucune** ligne de métier.

A · Le domaine devient pur

Aucune I/O, aucun import de SDK : il se lit comme la spécification du métier.

B · Les détails deviennent jetables

Changer de base, de mailer, de prestataire = écrire un adaptateur, pas toucher aux règles.

C · Les tests deviennent instantanés

Des faux en mémoire, des dates figées : la suite tourne en millisecondes.

La racine de composition : le seul fichier qui connaît les deux camps

```
// main.ts — 6 lignes, un seul endroit
const repo  = new MySqlCommandeRepo()                    // détail
const p   = new StripeGateway(process.env.STRIPE_KEY!)  // détail
const svc = new CommandeService(repo, p)                 // métier

// et dans les tests :
const svc = new CommandeService(new InMemoryRepo(), new FauxPaiement())
```

QUI POSSÈDE QUOI	OÙ ÇA VIT	RÈGLE
le port (l'interface)	`domain/ports.ts`	possédé par le **métier**, jamais par le fournisseur
l'adaptateur	`infra/<fournisseur>.ts`	seul fichier autorisé à importer le SDK
le montage	`main.ts` / `wiring.ts`	le **seul** endroit qui connaît les deux camps
les faux	`domain/testing/` ou les tests	un faux par port, écrit une fois

Le port est-il gros ?

Tant qu'il décrit **une frontière** et pas un fournisseur, sa taille n'est pas le sujet (c'est ISP qui tranche).

Combien d'adaptateurs ?

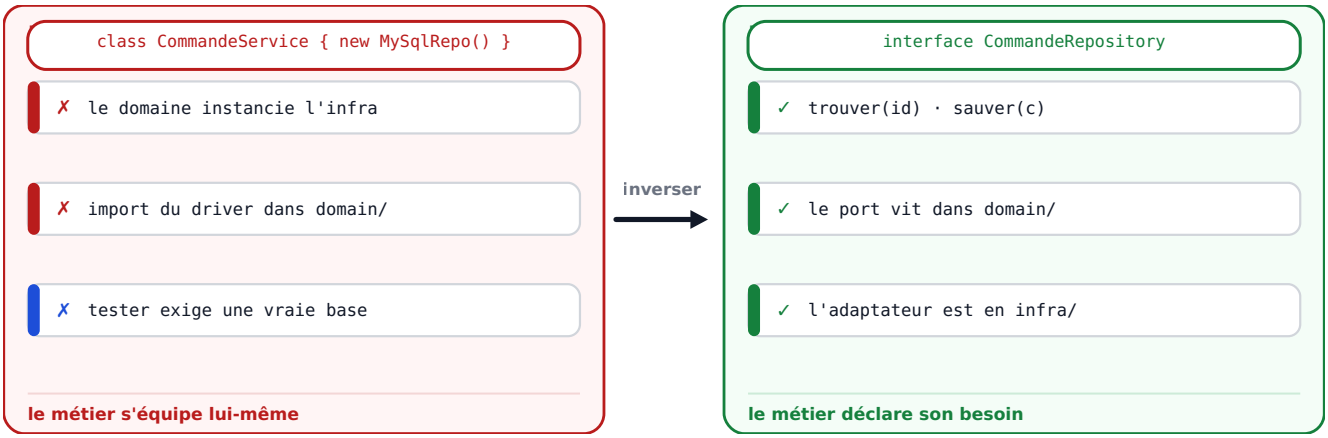
Au moins deux : le vrai et le faux. Un port à un seul adaptateur est souvent un paramètre déguisé.

Où sont les types ?

Dans le domaine, jamais dans `infra/` : un port qui importe l'infra n'inverse rien.

1 Le repository instancié

le new qui entre dans le domaine



AVANT

```
// domain/commande.service.ts
import { MySqlRepo } from "../infra/mysql-repo"

export class CommandeService {
  private repo = new MySqlRepo()

  valider(id: string) {
    const c = this.repo.trouver(id)
    if (!c) throw new Error("inconnue")
    c.valider()
    this.repo.sauver(c)
  }
}
```

APRÈS

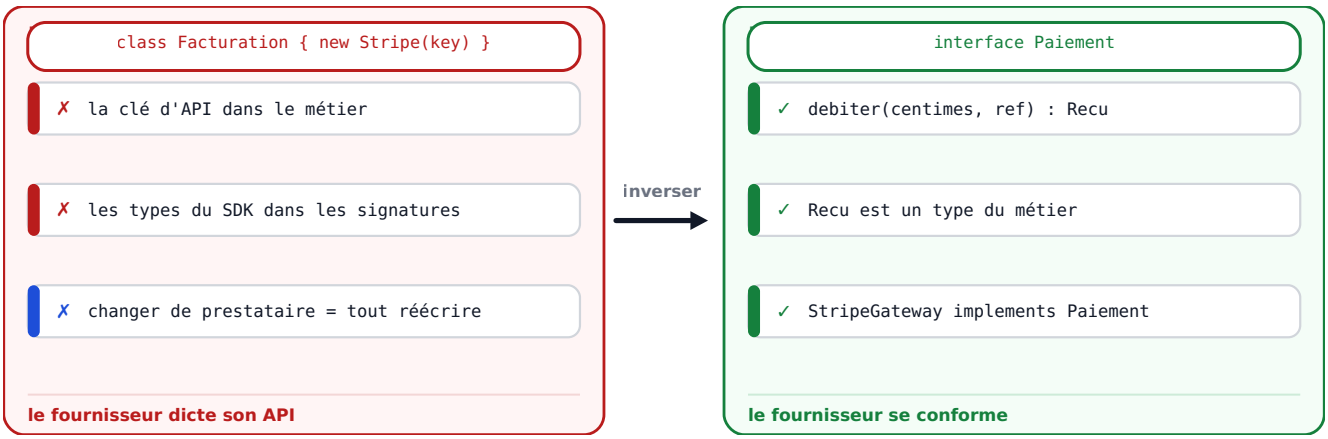
```
// domain/ports.ts — le port
export interface CommandeRepository {
  trouver(id: string): Commande | null
  sauver(c: Commande): void
}

// domain/commande.service.ts
export class CommandeService {
  constructor(private repo: CommandeRepository) {}
  // plus aucun import d'infra
}
```

À retenir : le corps de `valider()` ne change pas d'une ligne. Seul le **constructeur** change — et c'est tout le principe : le métier ne fait plus ses courses.

2 Le SDK de paiement

le fournisseur qui dicte son interface



AVANT

```
// domain/facturation.service.ts
import Stripe from "stripe"

class FacturationService {
  private stripe = new Stripe(process.env.KEY!)

  payer(f: Facture) {
    return this.stripe.charges.create({
      amount: f.centimes, currency: "eur" })
  }
}
```

APRÈS

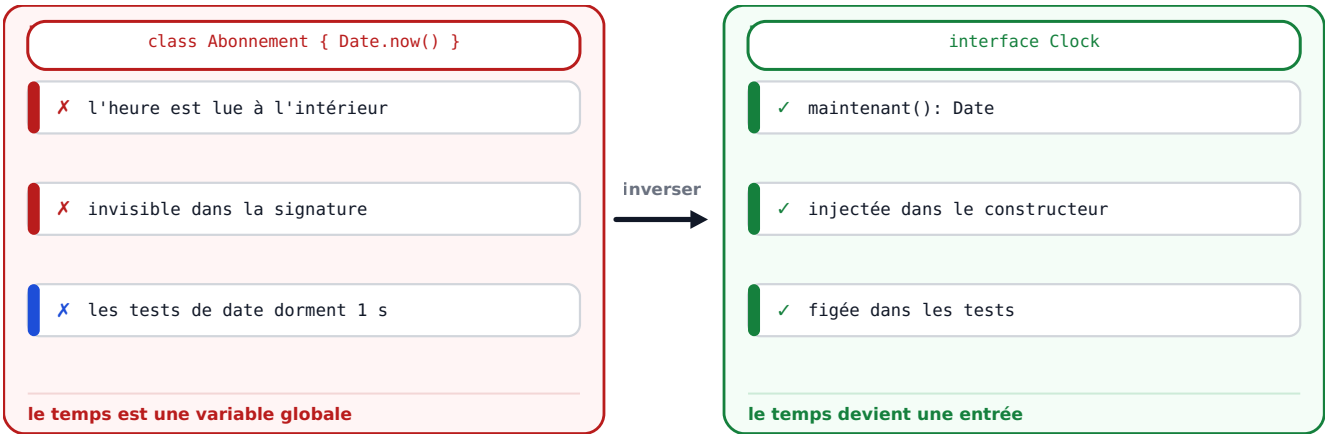
```
// domain/ports.ts
export interface Paiement {
  debiter(cents: number, ref: string): Promise<Recu>
}

class FacturationService {
  constructor(private p: Paiement) {}
  payer(f: Facture) {
    return this.p.debiter(f.centimes, f.id)
  }
}
// infra/stripe-gateway.ts : class StripeGateway
```

À retenir : le port ne doit **jamais** exposer un type du fournisseur (`Stripe.Charge`). S'il le fait, le détail a fui — et vous êtes mariés pour la vie.

3 L'horloge

Date.now() dans une règle métier



AVANT

```
class Abonnement {
  constructor(private fin: Date) {}

  estExpire(): boolean {
    return Date.now() > this.fin.getTime()
  }
}
// test : attendre… ou changer la date du CI
```

APRÈS

```
export interface Clock { maintenant(): Date }

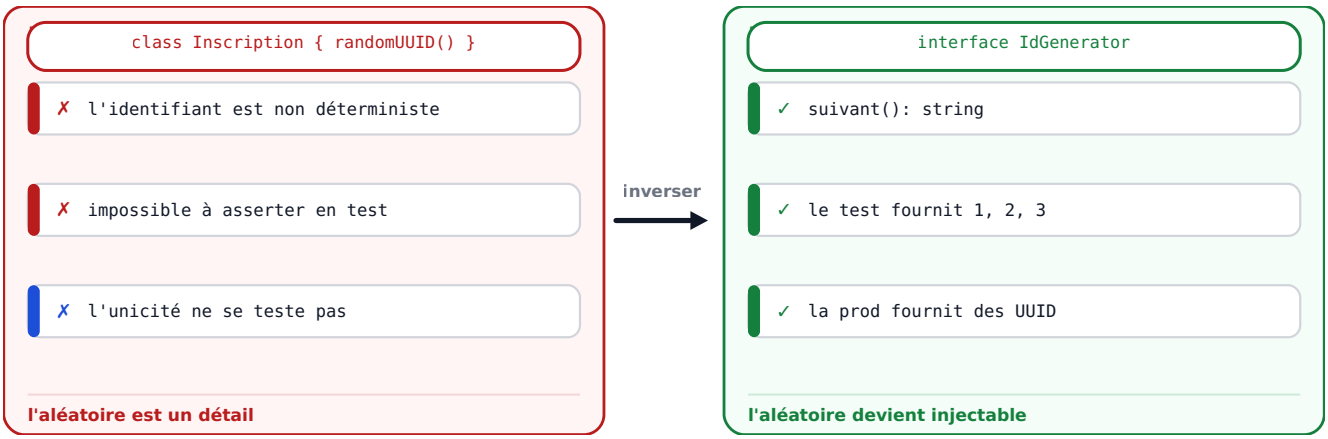
class Abonnement {
  constructor(private fin: Date,
              private clock: Clock) {}

  estExpire(): boolean {
    return this.clock.maintenant() > this.fin
  }
}
// test : { maintenant: () => new Date("2026-01-01") }
```

À retenir : c'est le port le plus **rentable** du principe : trois lignes de code, et tous les tests de date deviennent instantanés et déterministes.

4 L'identifiant aléatoire

le hasard caché dans un use case



AVANT

```
class Inscription {
  constructor(private repo: MembreRepo) {}

  executer(email: string) {
    const id = crypto.randomUUID()
    return this.repo.sauver({ id, email })
  }
}
// le test ne peut pas prévoir l'id
```

APRÈS

```
export interface IdGenerator { suivant(): string }

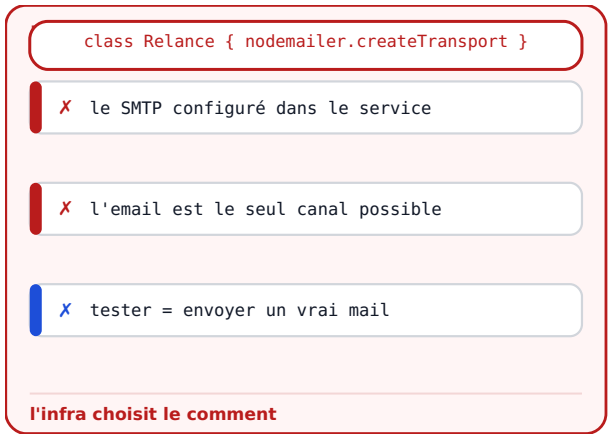
class Inscription {
  constructor(private ids: IdGenerator,
              private repo: MembreRepo) {}

  executer(email: string) {
    return this.repo.sauver(
      { id: this.ids.suivant(), email })
  }
}
// test : ids = { suivant: () => "id-1" }
```

À retenir : tout ce qui n'est pas **déterministe** — hasard, heure, identifiant, numéro de séquence — est une dépendance. Si le test ne peut pas le prévoir, c'est qu'il est caché.

5 L'envoi de mail

le canal en dur dans le service



AVANT

```
import nodemailer from "nodemailer"

class RelanceService {
  private mailer = nodemailer.createTransport({
    host: "smtp.exemple.fr" })

  relancer(f: Facture) {
    return this.mailer.sendMail({ to: f.email })
  }
}
```

APRÈS

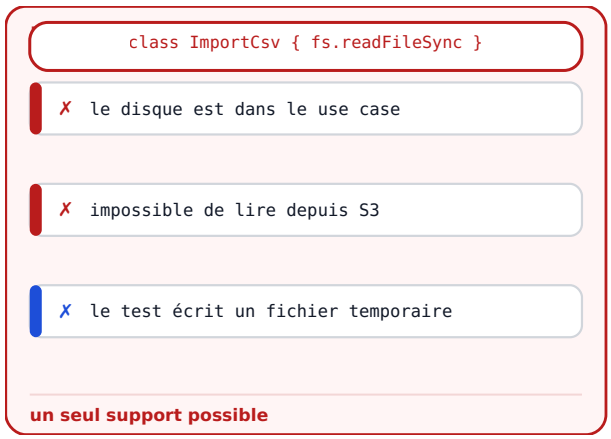
```
export interface Notificateur {
  envoyer(dest: string, msg: Message): Promise<void>
}

class RelanceService {
  constructor(private notif: Notificateur) {}
  relancer(f: Facture) {
    return this.notif.envoyer(f.email, relance(f))
  }
}
// SmtpNotif · SlackNotif · InMemoryNotif
```

À retenir : le métier décide **quand** notifier, l'infra décide **comment**. Le jour où l'on ajoute Slack, on écrit un adaptateur — pas une condition dans le service.

6 Le stockage de fichiers

le disque en dur dans le use case



AVANT

```
import fs from "node:fs"

class ImportCsv {
  traiter(chemin: string) {
    const c = fs.readFileSync(chemin, "utf8")
    return this.parser(c)
  }
}
// le jour où les fichiers sont sur S3…
```

APRÈS

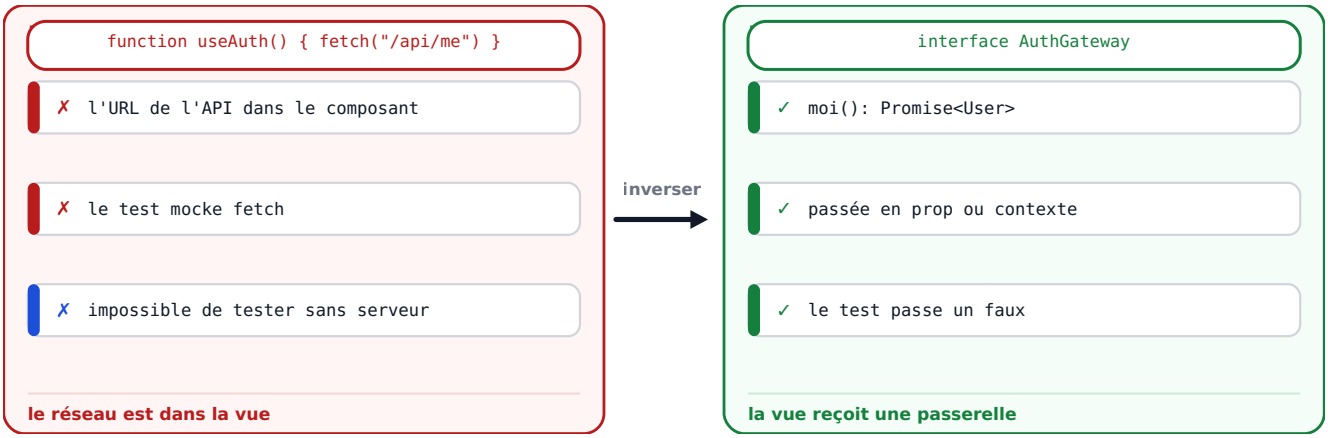
```
export interface Stockage {
  lire(cle: string): Promise<string>
  ecrire(cle: string, c: string): Promise<void>
}

class ImportCsv {
  constructor(private stockage: Stockage) {}
  async traiter(cle: string) {
    return this.parser(await this.stockage.lire(cle))
  }
}
```

À retenir : le même cas d'usage tourne sur disque, sur S3 ou en mémoire. Ce n'est pas de la sur-ingénierie : c'est ce qui permet de **tester** un import de fichier sans écrire de fichier.

7 Le fetch dans un composant

le réseau caché dans la vue



AVANT

```
function useAuth() {
  const [user, setUser] = useState(null)
  useEffect(() => {
    fetch("/api/me")
      .then(r => r.json())
      .then(setUser)
  }, [])
  return user
}
```

APRÈS

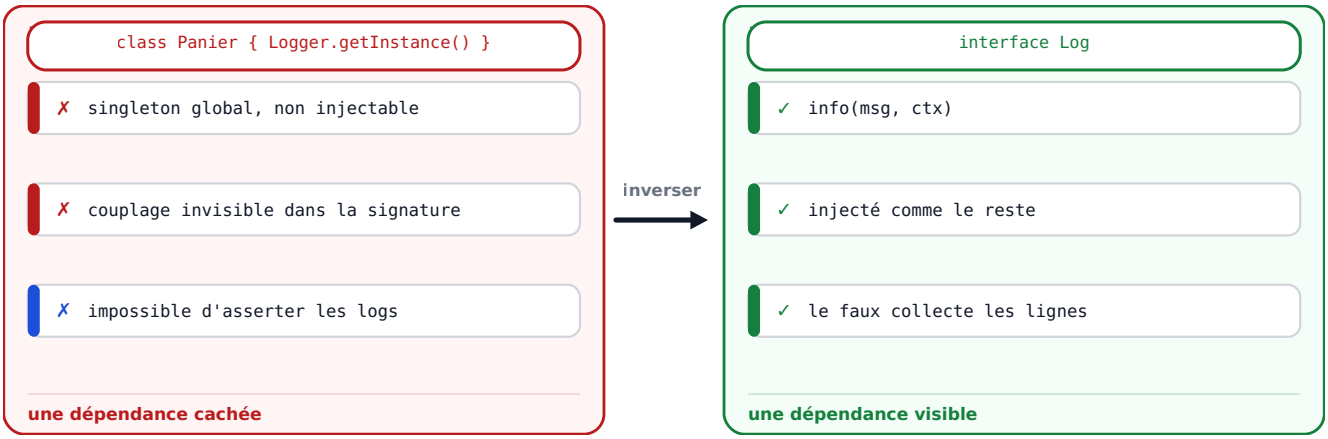
```
export interface AuthGateway {
  moi(): Promise<User>
}

function useAuth(gw: AuthGateway) {
  const [user, setUser] = useState(null)
  useEffect(() => {
    gw.moi().then(setUser)
  }, [gw])
  return user
}
// test : { moi: async () => ({ id: "1" }) }
```

À retenir : côté front, le port s'appelle une **passerelle**. Le composant ne sait plus qu'il y a un réseau — et le test ne mocke plus `fetch`, il passe un objet.

8 Le logger statique

le singleton qui remplace l'injection



AVANT

```
class PanierService {
  ajouter(a: Article) {
    Logger.getInstance().info("ajout")
    this.panier.push(a)
  }
}
// aucun constructeur : rien à injecter,
// donc rien à remplacer en test.
```

APRÈS

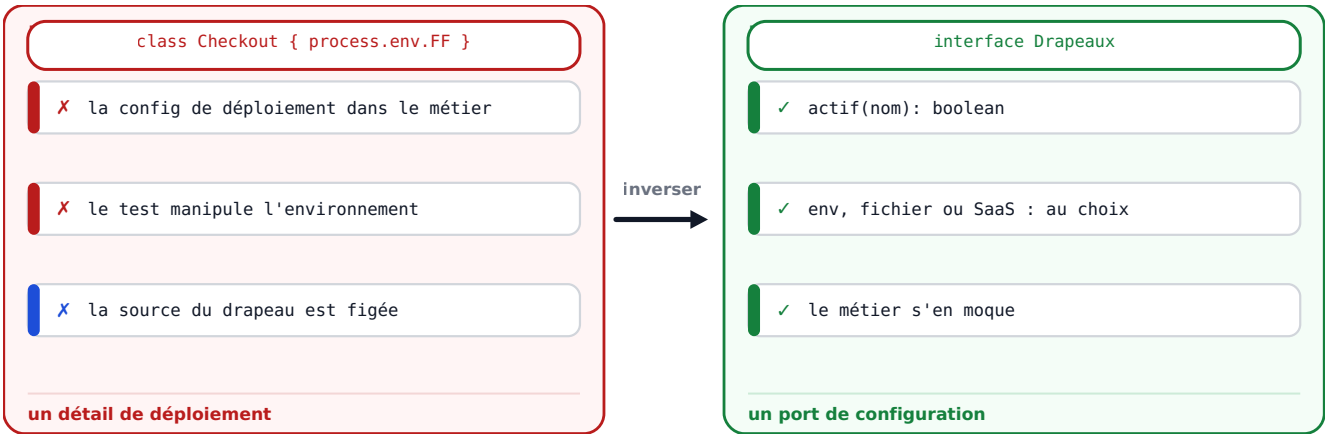
```
export interface Log {
  info(msg: string, ctx?: object): void
}

class PanierService {
  constructor(private log: Log) {}
  ajouter(a: Article) {
    this.log.info("ajout", { sku: a.sku })
    this.panier.push(a)
  }
}
// test : class FauxLog { lignes: string[] = [] }
```

À retenir : un singleton est l'anti-port : il est **global**, **mutable** et **invisible** dans la signature. S'il est vraiment unique dans l'app, c'est la racine de composition qui le crée — pas la classe qui l'appelle.

9 Les drapeaux de fonctionnalité

process.env dans une règle



AVANT

```
class Checkout {
  payer(c: Commande) {
    if (process.env.FF_NEW === "1")
      return this.payerV2(c)
    return this.payerV1(c)
  }
}
// le métier connaît le nom d'une variable
// d'environnement : c'est de l'infra.
```

APRÈS

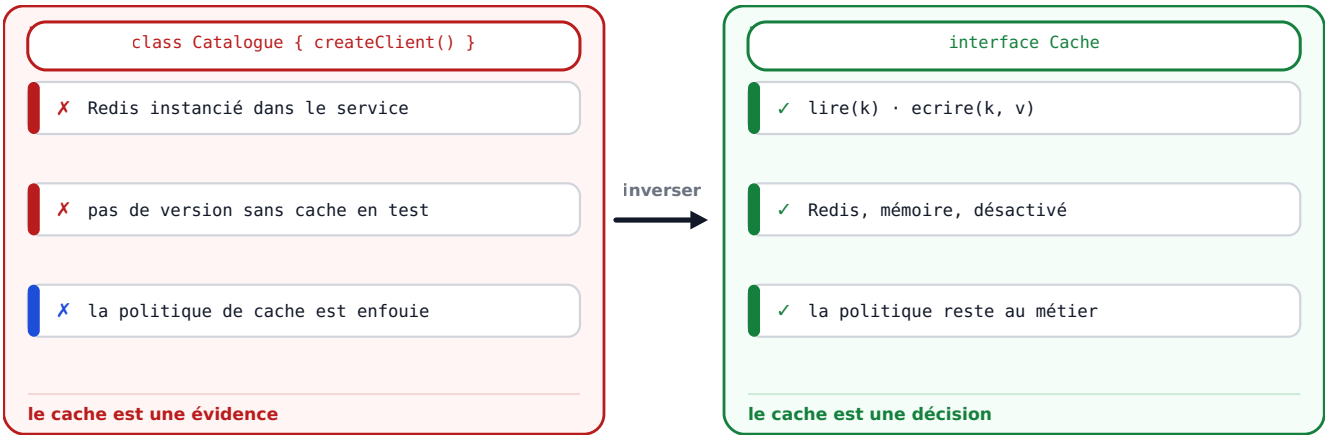
```
export interface Drapeaux {
  actif(nom: string): boolean
}

class Checkout {
  constructor(private flags: Drapeaux) {}
  payer(c: Commande) {
    return this.flags.actif("newPayment")
      ? this.payerV2(c)
      : this.payerV1(c)
  }
}
```

À retenir : une variable d'environnement lue dans un service, c'est un détail de **déploiement** qui entre dans la règle métier — et un test qui doit la manipuler.

10 Le cache

l'infrastructure qui décide de la performance



AVANT

```
import { createClient } from "redis"

class CatalogueService {
  private redis = createClient()

  async produit(id: string) {
    const hit = await this.redis.get(id)
    if (hit) return JSON.parse(hit)
    const p = await this.db.produit(id)
    await this.redis.set(id, JSON.stringify(p))
    return p
  }
}
```

APRÈS

```
export interface Cache {
  lire<T>(k: string): Promise<T | null>
  ecrire<T>(k: string, v: T): Promise<void>
}

class CatalogueService {
  constructor(private cache: Cache,
              private db: ProduitRepo) {}
  // CacheRedis · CacheMemoire · CacheDesactive
}
```

À retenir : le cache devient une **décision d'infrastructure** : on peut le désactiver en une ligne à la racine, ou le remplacer, sans toucher à la règle métier.

7 · TOUR D'HORIZON

Le reste de la boîte, en un coup d'œil

Neuf dépendances qu'on subit sans y penser : à gauche le détail en dur, à droite le port et ses adaptateurs.

File de messages
new Queue(redis://...)
	`import { Queue }`
	`new Queue(url)`
	`queue.add(job)`
		`interface File`
		`FileRedis`
		`FileMemoire (tests)`
			produire un job
			sans Redis

Appel HTTP
axios.get dans le use case
	`import axios`
	`axios.get(url)`
	`res.data`
		`interface HttpGet`
		`AxiosGet`
		`FauxGet`
			pas de serveur
			à lancer

Moteur de recherche
searchClient.index()
	`import { elastic }`
	`client.index(doc)`
	`client.search(q)`
		`interface Index`
		`IndexElastic`
		`IndexListe`
			rechercher dans
			une liste en test

Métriques
metrics.increment()
	`import { dd }`
	`dd.increment(k)`
	`dd.gauge(k, v)`
		`interface Metrics`
		`MetricsDatadog`
		`MetricsMemoire`
			asserter une
			métrique

Secrets
process.env.JWT_SECRET
	`const s = process`
	`.env.JWT_SECRET`
	`sign(s)`
		`interface Jeton`
		`secret au bord`
		`signataire injecté`
			le secret ne
			traverse plus

Génération PDF
new PDFDocument()
	`import PDFDocument`
	`const doc =`
	`new PDFDocument()`
		`interface GenerateurPdf`
		`PdfReel`
		`PdfTexte (test)`
			sortie texte
			en test

Persistance front
localStorage.getItem
	`localStorage`
	`.getItem(k)`
	`localStorage.set(k)`
		`interface Persistance`
		`PersistanceWeb`
		`PersistanceMemoire`
			le store tourne
			aussi en Node

WebSocket
new WebSocket(url)
	`const ws = new`
	`WebSocket(url)`
	`ws.send(m)`
		`interface Canal`
		`CanalWs`
		`CanalMemoire`
			le temps réel
			devient testable

Géocodage
geocoder.geocode(a)
	`import geocoder`
	`geocoder.lookup(a)`
	`.then(setCoords)`
		`interface Geocodeur`
		`GeocodeurGoogle`
		`GeocodeurFixe`
			pas de quota
			consommé en test

A · Le réflexe

Un new ou un `import` de SDK dans `domain/` : le détail est entré.

B · La question

« Puis-je tester cette règle sans base, sans réseau, sans horloge réelle ? » Si non, il manque un port.

C · Le gain caché

Un domaine pur se lit comme une spécification : les gens du métier peuvent le relire, et le relisent.

Le fil rouge de ces neuf cas : on n'inverse pas « pour faire propre ». On inverse **parce qu'un détail nous empêche de tester, de changer ou de comprendre**. Pas de douleur, pas de port.

DIP n'est pas un conteneur d'injection

C'est le contresens n°1 : on branche un outil d'injection et on croit avoir inversé les dépendances. La flèche, elle, n'a pas bougé.

CE QU'ON CROIT FAIRE — le conteneur magique

```
@Injectable()
class CommandeService {
constructor(private repo: MySqlRepo) {}
}
// et MySqlRepo fait new Driver()
```

- ✗ l'injection ne change rien au sens
- ✗ le domaine importe toujours le driver
- ✗ le test a toujours besoin d'une base

on a branché un outil, pas inversé une flèche

CE QUE ÇA VEUT DIRE — le port et l'adaptateur

```
// domain/ports.ts
interface CommandeRepository { … }
// infra/mysql-repo.ts
class MySqlRepo implements CommandeRepository
// main.ts : le seul qui connaît les deux
```

- ✓ le métier possède l'interface
- ✓ l'infra s'y conforme
- ✓ le montage est au bord

c'est le sens de la dépendance qui a changé

Ce qu'on inverse — et ce qu'on laisse tranquille

À INVERSER

- ✓ base de données, fichier, réseau
- ✓ heure, hasard, identifiants
- ✓ fournisseurs (paiement, mail, SMS)
- ✓ configuration de déploiement

À NE PAS INVERSER

- ✗ Math, JSON, Array, String
- ✗ les value objects (Montant, Email)
- ✗ les types du langage
- ✗ les fonctions pures de votre domaine

LE PIÈGE	CE QU'IL RÉVÈLE	LE GESTE
`@Injectable()` sur une classe qui fait new `MySql()`	l'outil a changé, pas le sens	extraire le port **dans le domaine**
`container.get("repo")` (service locator)	une dépendance cachée, pas injectée	passer par le constructeur
tout injecter, même `Math`	de l'indirection sans bénéfice	n'inverser que ce qui **varie**
un port qui expose `Stripe.Charge`	le détail a fui dans l'abstraction	des types du **métier**
un port qui change à chaque version du SDK	c'est un adaptateur déguisé	stabiliser le vocabulaire

A · L'outil n'inverse rien

Un conteneur **câble** des objets ; il ne change pas le sens de la dépendance. On peut très bien injecter un détail subi.

B · L'inversion est dans le code

Elle se voit dans les `import` : si `domain/` n'importe rien d'`infra/`, c'est fait.

C · Le locator n'est pas un port

`container.get()` remplace new par une dépendance **cachée** : ni testable, ni lisible.

Le conteneur fait quoi ?

Il instancie et câble. Il ne décide pas qui possède l'interface — c'est toi, dans le code.

Comment le vérifier ?

Supprime le dossier `infra/` : ce qui casse est ce qui dépend encore du détail.

Et le service locator ?

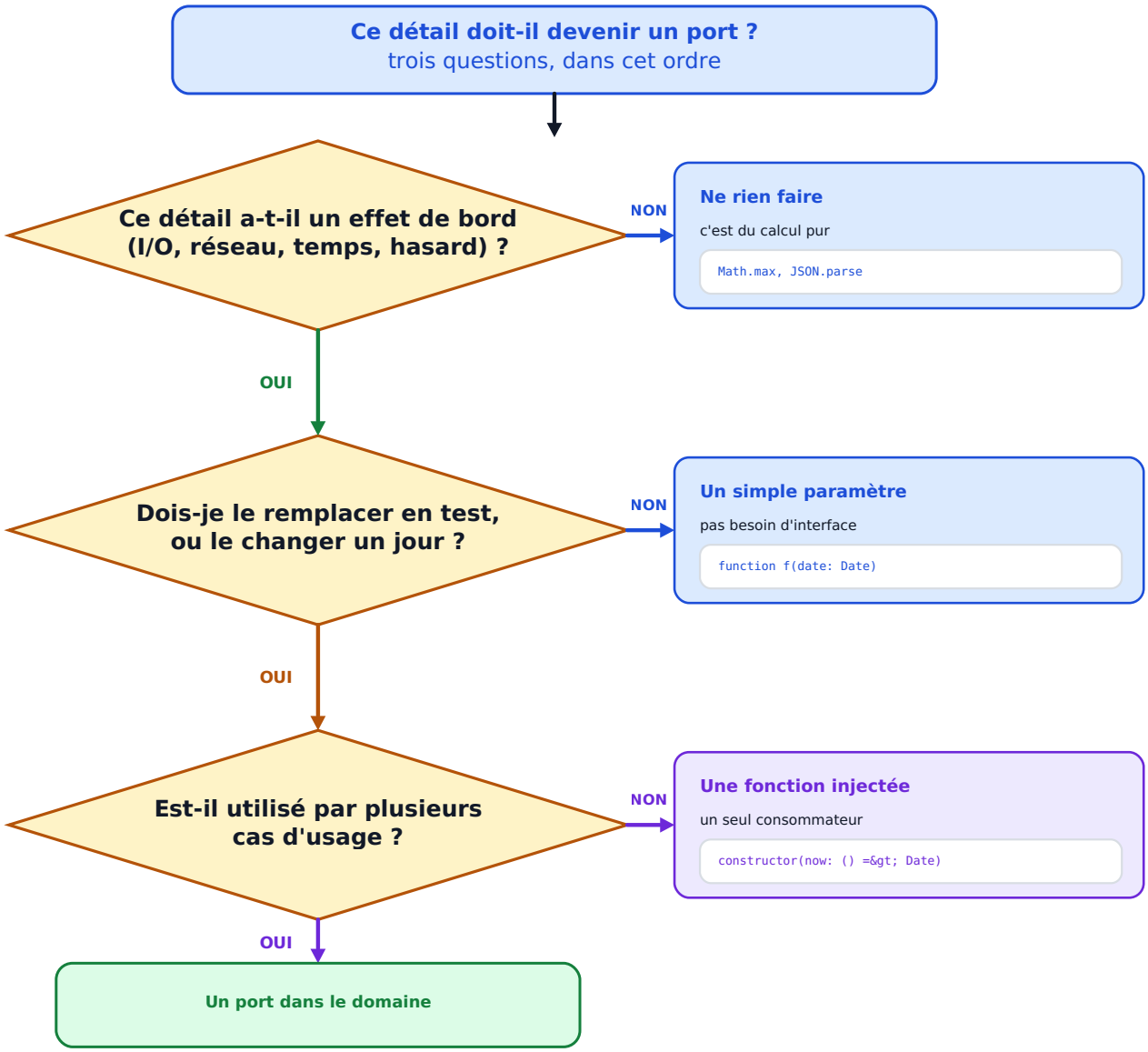
`container.get()` cache la dépendance au lieu de la déclarer : c'est l'inverse de DIP.

La question qui tranche : si je supprime le dossier `infra/`, qu'est-ce qui casse ? Si le domaine casse, la flèche est encore dans le mauvais sens — quel que soit le nombre d'annotations.

9 · DÉCIDER

Ce détail doit-il devenir un port ?

Trois questions, dans cet ordre. La première élimine tout ce qui est du calcul pur.



Dans le doute : écris le test sans le détail.

S'il faut monter une base ou attendre une seconde, le port manque.

Repository → port

le domaine possède l'interface

Date.now → Clock

3 lignes, tous les tests débloqués

Math.max → rien

du calcul pur : on n'inverse pas

A · Un port par méthode ?

Non : un port par **frontière**. Trois méthodes de la même frontière vivent ensemble dans le même port.

B · Un port qui ne sert qu'au test ?

C'est déjà une raison suffisante — mais si personne ne peut l'implémenter autrement, un paramètre suffit.

C · Un port pour une fonction pure ?

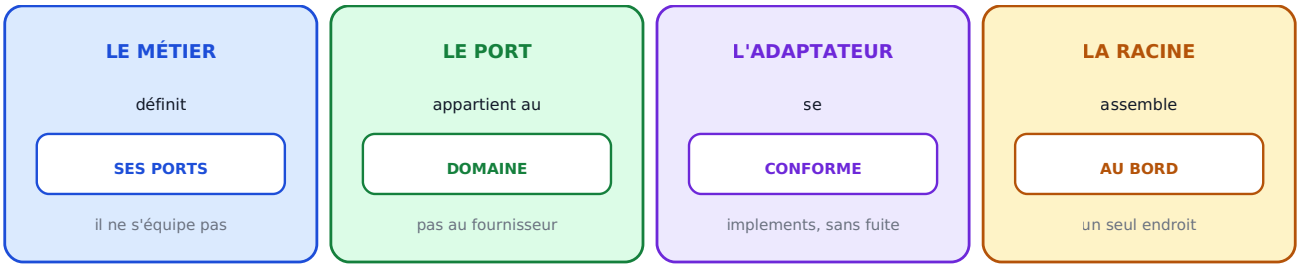
Non. `Math.round`, `JSON.parse`, un `Montant.plus()` ne sont pas des frontières.

Le raccourci : un port n'est pas « une interface de plus » — c'est une **frontière**. Si tu ne peux pas nommer un second adaptateur (un faux, un autre fournisseur), c'est probablement un simple paramètre.

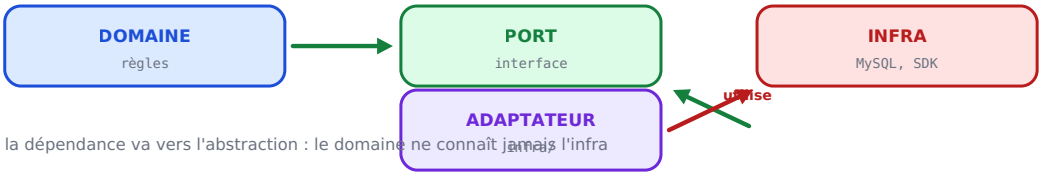
10 · RÉVISER

Le principe en une image, et la flèche qui le résume

Si tu retiens une page, c'est celle-ci.



Le réflexe visuel : la flèche



En une phrase ?

Le métier et l'infra dépendent d'abstractions ; jamais l'un de l'autre.

Qu'est-ce qui s'inverse ?

Le sens de la dépendance — et la propriété de l'interface, qui passe au métier.

À qui appartient l'interface ?

Au module de haut niveau, celui qui l'utilise. C'est le cœur du principe.

Les signaux ?

Un new technique dans domain/ · un import de SDK · un test qui exige une base · un singleton getInstance() · un process.env dans une règle.

Les 3 gestes ?

Extraire le port côté client → pousser l'adaptateur derrière → injecter à la racine.

DIP vs ISP ?

ISP règle la taille de l'abstraction ; DIP règle qui la possède et le sens de la flèche.

DIP vs OCP ?

OCP demande un point d'extension ; DIP le place côté métier, sous forme de port.

DIP vs LSP ?

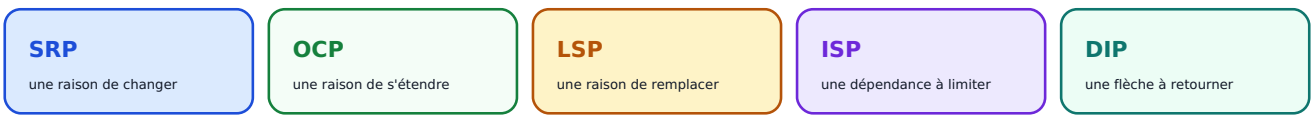
Un port est un contrat : chaque adaptateur doit le tenir. LSP s'applique aux adaptateurs.

Le piège n°1 ?

Croire que le conteneur d'injection fait le travail. Il câble, il n'inverse pas.

CHECKLIST AVANT DE DIRE « C'EST INVERSÉ »	SI NON →
Le domaine compile-t-il **sans** le dossier infra ?	la dépendance est encore subie
Aucun new technique dans `domain/` ?	extraire un port ou injecter un paramètre
Le port n'expose-t-il **aucun** type du fournisseur ?	le détail a fui : renommer les types
Le use case tourne-t-il avec des faux en mémoire ?	il manque au moins un port
Existe-t-il **un seul** endroit qui assemble tout ?	disperser le montage dans la racine

Les cinq principes côte à côte



> **Et si tu oublies tout le reste :** demande-toi **qui possède l'interface**. Si c'est le fournisseur, tu n'as pas inversé : tu as ajouté une couche. Si c'est ton métier, la flèche est bonne.

DIP dans SOLID

Les cinq principes se répondent : DIP est celui qui place les abstractions **du bon côté**.

DIP est le principe qui rend les quatre autres possibles



OCP → DIP	un point d'extension est un port : DIP décide où il vit (côté métier) et qui le possède.
LSP → DIP	un port est un contrat ; chaque adaptateur doit le tenir. Une violation de LSP sur un adaptateur casse le métier.
ISP → DIP	ISP règle la taille du port ; DIP règle sa propriété. Petit ET possédé par le client.
SRP → DIP	séparer les responsabilités, c'est déjà séparer ce qui touche le monde de ce qui calcule.

La règle qui résume : une dépendance doit toujours pointer vers ce qui change le moins (stabilité).

PRINCIPE	LA QUESTION	CE QU'IL DÉCOUPE
SRP	« quelles responsabilités ? »	les raisons de changer
OCP	« puis-je ajouter sans modifier ? »	les points d'extension
LSP	« puis-je remplacer sans casser ? »	les contrats et leurs promesses
ISP	« de quoi ce client a-t-il besoin ? »	la taille des dépendances
DIP	« qui possède l'abstraction, et dans quel sens ? »	le **sens** des dépendances

L'ordre de lecture : SRP découpe le code, ISP ce que le client en voit, DIP décide **où vit l'interface** et **qui en dépend**. Sans DIP, OCP reste un vœu : on ne peut pas étendre proprement un module qui connaît ses détails.

PRINCIPE	CE QU'IL NE DIT PAS	LE CONTRESENS COURANT
SRP	qu'une classe doit être petite	« une méthode par classe »
OCP	qu'il faut une abstraction pour tout	« une interface par classe »
LSP	qu'il faut hériter	« un sous-type = extends »
ISP	que les interfaces doivent être petites	« une interface par méthode »
DIP	qu'il faut un conteneur d'injection	« @Injectable = inversé »

A · Sans DIP

OCP devient un vœu : on ne peut pas ajouter un cas sans modifier un module qui connaît ses détails.

B · Sans DIP, pas de tests

Le métier ne se teste qu'avec le monde entier : la suite devient lente, puis elle disparaît.

C · Avec DIP

Le domaine se lit comme une spécification, se teste en millisecondes, et survit à trois changements de fournisseur.

Injecter, sans en faire une religion

Un paramètre suffit souvent ; l'interface n'arrive que quand la frontière est réelle.

Trois façons d'injecter — et une seule bonne par défaut

1 Constructeur

obligatoire, visible

```
constructor(private r: Repo)
```

→ le défaut

2 Paramètre de méthode

ponctuel, explicite

```
valider(id, clock: Clock)
```

→ quand l'appel varie

3 Setter / propriété

optionnel, mutable

```
svc.clock = …
```

→ à éviter

Les 5 refactorings DIP, du plus simple au plus profond

1 **Remplacer new par un paramètre**
une ligne, zéro interface

2 **Extraire le port côté client**
l'interface naît dans domain/

3 **Déplacer le code en adaptateur**
le SDK reste dans infra/

4 **Écrire le faux en mémoire**
la preuve que le port suffit

5 **Centraliser le montage**
une racine de composition

```
// main.ts — la racine de composition, seul fichier qui connaît les deux camps
const repo = new MySqlCommandeRepo(); const clock = new SystemClock()
const svc = new CommandeService(repo, clock); app.use(svc)
```

LE REFACTORING	QUAND	RISQUE SI ON SAUTE L'ÉTAPE
remplacer new par un paramètre	la dépendance est stable	on abstrait pour rien
extraire le port côté client	le détail est instable ou coûteux	le port ressemble au SDK
déplacer le code en adaptateur	le SDK est encore dans le métier	le détail fuit par les types
écrire le faux en mémoire	le port est trop gros	on ne sait pas s'il suffit
centraliser le montage	le new est dispersé	12 endroits à connaître

Faut-il un port pour tout ?

Non : un seul adaptateur prévu et une dépendance stable — un paramètre suffit largement.

Où écrire l'interface ?

Dans le module qui l'utilise. Si tu l'écris dans `infra/`, c'est déjà perdu.

Qui appelle le constructeur ?

La racine, et personne d'autre. Un new dans un service = une dépendance re-subie.

L'ordre compte. Si tu écris l'adaptateur d'abord, tu inventes une interface à l'image du SDK — et tu n'as rien inversé : tu as ajouté une couche. Le port s'écrit **avec les mots du métier**, avant de savoir qui l'implémentera.

LE PIÈGE D'INJECTION	CE QUI SE PASSE	LE GESTE
injecter une classe concrète	`private repo: MySqlRepo` — le test doit fournir un MySQL	injecter le **port**
un constructeur à 7 paramètres	le service fait trop de choses	regrouper : c'est SRP qui parle
un port qui ne sert qu'à un test	indirection sans bénéfice	un simple paramètre de méthode
un `setter` pour tout	dépendances optionnelles et mutables	constructeur, obligatoire

13 · S'ENTRAÎNER

Cinq extraits, cinq gestes

Pour chaque extrait : quel détail, quel effet de bord, quel geste ?

À toi : cinq extraits, cinq gestes

1

```
class A {
private db =
new Pg()
run() { … }
}
```

symptôme
le domaine instancie

geste
paramètre + port

2

```
import axios
axios.get(url)
.then(r =&gt; r.data)
```

symptôme
réseau dans le use case

geste
port HttpGet

3

```
if (process.env
.FF === '1')
return v2()
```

symptôme
config dans le métier

geste
port Drapeaux

4

```
const d = new Date()
if (d &gt; fin) …
```

symptôme
l'heure est cachée

geste
port Clock

5

```
let repo: Repo
repo = new MySql()
// 12 fois dans
// le projet
```

symptôme
montage dispersé

geste
une racine

Solutions en bas de page — cherche d'abord, c'est là que ça rentre.

Vrai ou faux ?

Un conteneur d'injection suffit à appliquer DIP.	**FAUX**	— il câble, il n'inverse pas
L'interface doit vivre du côté du fournisseur.	**FAUX**	— elle appartient au métier
On peut inverser une dépendance avec un simple paramètre.	**VRAI**	— pas besoin d'interface partout
Un port peut exposer les types du SDK.	**FAUX**	— le détail aurait fui
Un service locator est une forme d'injection.	**FAUX**	— c'est une dépendance cachée
Un domaine qui compile sans infra a la bonne flèche.	**VRAI**	— c'est le test décisif

Solutions : 1 → passer le repository en paramètre puis extraire le port · 2 → HttpGet + AxiosGet / FauxGet · 3 → port Drapeaux (env, fichier ou SaaS) · 4 → port Clock (3 lignes, tous les tests débloqués) · 5 → une seule racine de composition.

Corrigé n°3 — le drapeau devient un port

```
export interface Drapeaux { actif(n: string): boolean }

class Checkout {
  constructor(private flags: Drapeaux) {}
  payer(c: Commande) {
    return this.flags.actif("newPayment")
      ? this.payerV2(c) : this.payerV1(c)
  }
}
```

Corrigé n°5 — une seule racine

```
// main.ts
const repo = new MySqlCommandeRepo()
const svc  = new CommandeService(repo)

// et nulle part ailleurs :
// aucun new MySql... dans domain/
// grep -rn "new MySql" src/ → 1 ligne
```

Exercice 6

Port `HttpGet` ou `Tarifs` ?
→ **Tarifs** : le port parle métier.

Exercice 7

Une seule lecture de `new Date()` ?
→ un paramètre `now: Date` suffit.

Exercice 8

Deux clients, un gros port ?
→ non : deux ports. C'est ISP qui tranche.

AUTO-ÉVALUATION	OUI	NON
mon domaine compile sans le dossier infra	→ la flèche est bonne	→ il reste un import à sortir
aucun `new` technique dans `domain/`	→ c'est propre	→ paramètre ou port manquant
mes tests métier tournent en moins d'une seconde	→ c'est inversé	→ il manque un faux en mémoire
un seul fichier connaît les deux camps	→ racine de composition	→ le `new` est dispersé
mes ports n'exposent aucun type de SDK	→ pas de fuite	→ renommer les types du port

14 · IMPRIMER

La fiche mémo

Une page, à coller à côté de l'écran : la règle, les leviers, le test, les signaux.

La fiche à imprimer : tout DIP sur une page

LA RÈGLEMENT

Le métier et l'infra dépendent d'abstractions ; **jamais l'un de l'autre.**

→ le port appartient au métier

LES 4 LEVIERS

- ✓ inverser la flèche
- ✓ le détail s'y conforme
- ✓ composer au bord
- ✓ abstraction stable

LE TEST DÉCISIF

`supprime infra/`

→ le domaine compile ?

→ le test tourne sans base ?

deux oui = c'est inversé

Signaux → gestes

✗ `new MySql() dans domain/`	→	✓ **port + adaptateur**
✗ `import Stripe dans le service`	→	✓ **port Paiement, types du métier**
✗ `Date.now(), randomUUID()`	→	✓ **port Clock, IdGenerator**
✗ `process.env dans une règle`	→	✓ **paramètre ou port Drapeaux**
✗ `Logger.getInstance()`	→	✓ **constructeur + interface Log**
✗ `new` à 12 endroits	→	✓ **une racine de composition**

L'arbre de décision (page 14), en trois lignes

1. Effet de bord (I/O, réseau, temps, hasard) ?	**→ non : ne rien faire**
2. Dois-je le remplacer en test ou un jour ?	**→ non : un simple paramètre**
3. Utilisé par plusieurs cas d'usage ?	**→ oui : un port dans le domaine**

DIP en une phrase ?

Le métier et l'infra dépendent d'abstractions ; jamais l'un de l'autre.

Le test ?

Supprime `infra/` : si le domaine compile et que ses tests passent, c'est inversé.

Le geste ?

Extraire le port côté client → pousser l'adaptateur → injecter à la racine.

LES 8 GESTES EXPRESS	OÙ
chercher les `new` dans `domain/` et les remplacer par un paramètre	4
écrire l'interface dans le module qui l'utilise	2
nommer le port avec le vocabulaire du métier	2
déplacer le code du SDK dans `infra/`	7
écrire un faux en mémoire par port	10
figer l'heure et le hasard pour les tests	9
un seul `main.ts` qui assemble	6
vérifier que `domain/` n'importe rien d'`infra/`	3

Le réflexe à désapprendre

```
class Service {
  private db = new Pg()
  private mail = new Smtp()
}
// le service se sert tout seul :
// impossible de le tester, de le
// déplacer, de le comprendre seul.
```

Le squelette d'un service inversé

```
class Service {
  constructor(
    private db: CommandeRepository,   // port
    private mail: Notificateur,       // port
  ) {}
}
// aucun import d'infra dans ce fichier
// les adaptateurs arrivent par main.ts
```